
CS5340 Assignment 2:

Junction Tree Algorithm and Parameter Learning

1 Overview

This assignment has two parts. In Part 1, you will construct a junction tree from a Markov random field and perform inference on the junction tree using the sum-product algorithm. In Part 2, you will learn the parameters of linear-Gaussian Bayesian networks from complete observations.

References: Lectures 4, 5, and 6

Honour Code. This coding assignment constitutes **15%** of your final grade in CS5340. Note that plagiarism will not be condoned! You may discuss with your classmates and check the internet for references, but you **must not** submit code or a report copied directly from other sources.

2 Submission Instructions

Submit one zip archive containing the following:

- **Part 1 source code:** the files `jt_construction.py`, `main.py`, and `factor_utils.py` inside `part1/`.
- **Part 2 source code:** `part2/main.py`.
- **Part 1 report (report1.pdf).** Describe your implementation in no more than one page.
- **Part 2 report (report2.pdf).** Describe your implementation and derivations in no more than one page.

Please indicate clearly your name and student number (the one that looks like A1234567X) in both reports and at the top of your source code. Name the archive `A1234567X_lab2.zip`, replacing the example with your student number. Its structure should be:

```
A1234567X_lab2.zip
|-- part1/
|-- part2/
|-- report1.pdf
'-- report2.pdf
```

Submit your assignment by **27 September 2026** to Canvas. 25% of the total score will be deducted for each day of late submission.

3 Getting Started

This assignment requires Python 3.5 or later. Install the required packages from within each part using:

```
pip install -r requirements.txt
```

If you have installation issues, post them in the forum so that other students or the instructors can help accordingly.

4 Test Cases

Sample datasets and their expected outputs are provided to help you debug your implementation. For Part 1, inputs are in `data/inputs`, ground truth is in `data/ground-truth`, and reference answers are in `data/answers`. For Part 2, observations are in `data/observations`, generating parameters are in `data/gt-parameters`, and reference answers are in `data/answers`.

Ground truth and reference answers may differ slightly because of numerical precision or the finite number of observations. During grading, your code will be evaluated on hidden test cases in addition to the provided validation cases.

5 Part 1: Junction Tree Algorithm

The sum-product algorithm used in Assignment 1 works directly on tree-like graphs, but not on loopy networks. To make a loopy Markov random field amenable to sum-product inference, we first convert it into a junction tree.

5.1 Basic Factor Operations

Implement the following functions in `factor_utils.py`:

- **`factor_product()`**: Compute the product of two factors.
- **`factor_marginalize()`**: Sum over the indicated variable or variables and return the resulting factor.
- **`factor_evidence()`**: Reduce a factor to the entries consistent with the observations and remove the observed variables from its scope. Return an empty factor if all variables in its scope are observed.

These operations use the custom `Factor` class from Assignment 1. Note that `factor_evidence()` is different from `observe_evidence()` in Assignment 1 because observed variables are removed from the returned factor.

5.2 Constructing the Junction Tree

Complete the following steps in `jt_construction.py`:

- Build the junction-tree graph structure by forming maximal cliques and creating edges between cliques in `_get_jt_clique_and_edges()` [**3 points**].
- Assign factors from the original graph to the cliques in `_get_clique_factors()` [**1 point**].

Evidence variables will also be provided. Update both the graph structure and the factors after incorporating the evidence.

5.3 Inference on the Junction Tree

Once the junction tree has been constructed, complete the following in `main.py`:

- Use sum-product message passing to compute the clique potentials [**2 points**]. Implement this in `_get_clique_potentials()`.
- Retrieve every query-node marginal, such as $p(X_2 \mid X_E)$ and $p(X_3 \mid X_E)$, from the clique potentials in `_get_node_marginal_probabilities()` [**2 points**].

The second step includes one point for an efficient solution. As a hint, cliques have varying sizes. You may create additional helper functions inside the designated comment blocks.

6 Part 2: Parameter Learning

In this part, observations are complete and the graph structure is known. Let $X_1, \dots, X_V$ be the random variables and let N complete observations be given. For each node X_u , learn the maximum-likelihood estimate of its local parameters θ_u in $p(x_u \mid x_{\pi_u}, \theta_u)$.

The local conditional distribution is a linear-Gaussian model:

$$p(x_u \mid x_{\pi_u}, \theta_u) = \mathcal{N}\left(\sum_{c \in \pi_u} w_{uc} x_c + w_{u0}, \sigma_u^2\right), \quad (1)$$

or, equivalently,

$$p(x_u \mid x_{\pi_u}, \theta_u) = \frac{1}{\sqrt{2\pi\sigma_u^2}} \exp\left[-\frac{(x_u - (\sum_{c \in \pi_u} w_{uc} x_c + w_{u0}))^2}{2\sigma_u^2}\right]. \quad (2)$$

Because the observations are complete, parameters can be learned separately for each node by maximising its local log-likelihood:

$$\hat{\theta}_u = \operatorname{argmax}_{\theta_u} \sum_{n=1}^N \log p(x_u^{(n)} \mid x_{\pi_u}^{(n)}, \theta_u) \quad (3)$$

$$= \operatorname{argmax}_{\theta_u} \sum_{n=1}^N \left[-\frac{1}{2} \log(2\pi\sigma_u^2) - \frac{1}{2\sigma_u^2} \left(x_u^{(n)} - \left(\sum_{c \in \pi_u} w_{uc} x_c^{(n)} + w_{u0} \right) \right)^2 \right]. \quad (4)$$

Taking derivatives gives linear equations for the weights and a closed-form estimate of the variance. Implement:

- **_learn_node_parameter_w()**: Solve for $w_{u0}, \dots, w_{uC}$ [**3 points**].
- **_learn_node_parameter_var()**: Using the learned weights, solve for σ_u^2 [**1 point**].
- **_get_learned_parameters()**: Call the preceding functions for every node and return all learned parameters [**2 points**].

The derivations in your report are also important [**1 point**]. Test cases 1 and 2 use the single-node example from Lecture 6; cases 3 and 4 use the three-node example with different numbers of observations. Hidden cases will contain more complex graphs, so your implementation should scale. No parameter evidence will be introduced in the test cases.

Hint: You may create additional helper functions inside the designated comment blocks.